

เอกสารประกอบรายงาน · ระยะที่ 2

แผนภาพระบบของโครงการพัฒนาระบบตรวจจับ เครื่องมือผ่าตัดและบริเวณแผลด้วยปัญญาประดิษฐ์

ชุดแผนภาพ 11 รูป ครอบคลุมทั้งโปรเจกต์ — ตั้งแต่สถาปัตยกรรมซอฟต์แวร์ การไหลของข้อมูล มุมมองผู้ใช้ ไปจนถึงอัลกอริทึมและโครงสร้างข้อมูล ทุกรูปเขียนตามหลักสัญลักษณ์มาตรฐาน (UML / DFD / Flowchart) และอ้างอิงจากซอร์สโค้ดจริงในระบบ ไม่ใช่การออกแบบบนกระดาษ

11 แผนภาพ	6 หมวด	อ้างอิงโค้ดจริง ณ ส.ค. 2569	YOLOv11 · FastAPI · 29 endpoints
-----------	--------	-----------------------------	----------------------------------

สารบัญ

ก · ภาพรวมระบบ

- 1 สถาปัตยกรรมระบบ
- 2 การติดตั้งใช้งาน

ข · การไหลของข้อมูล

- 3 Context Diagram
- 4 DFD ระดับที่ 1

ค · มุมมองผู้ใช้

- 5 Use Case Diagram

ง · พฤติกรรมเชิงเวลา

- 6 ลำดับงานสกัดเฟรม
- 7 ลำดับงาน Label Assist

จ · อัลกอริทึม

- 8 ผังงานสกัดเฟรม
- 9 วงจรกำกับข้อมูล

ฉ · ข้อมูลและสถานะ

- 10 แผนภาพสถานะ
- 11 โครงสร้างข้อมูล

ภาคผนวก

- ใช้รูปไหนในบทใด

รายงานผลการดำเนินงานระยะที่ 1 อธิบายโครงการนี้ในฐานะ **งานวิจัยโมเดล** คือ เตรียมข้อมูลจากวิดีโอ 12 คลิป สกัดได้ 120 เฟรม กำกับข้อมูลครบทุกภาพ แล้วเทรน YOLOv11 จนได้ mAP@50 68.1% / Precision 80.5% / Recall 73.1%

แต่ปัจจุบันโครงการได้พัฒนา **เครื่องมือเว็บแอปพลิเคชัน** ขึ้นมารองรับกระบวนการทั้งหมดสถานะของงานจึง เปลี่ยนจาก “งานวิจัยโมเดล” เป็น “**ระบบซอฟต์แวร์ที่มีปัญญาประดิษฐ์เป็นส่วนประกอบ**” ซึ่งต้องใช้แผนภาพ คนละชุดในการอธิบาย — แผนภาพสายข้อมูล/โมเดลอย่างเดียวไม่เพียงพอ ต้องมีแผนภาพระดับระบบ (สถาปัตยกรรม, Use Case, Sequence) เพิ่มเข้ามาด้วย เอกสารนี้จึงรวบรวมแผนภาพทั้งสองด้านไว้ครบในชุดเดียว

วัตถุเป้าหมาย 5 ประเภท

☐ เข็มเย็บแผล needle

☐ นิ้วมือผู้ปฏิบัติงาน finger

☐ คีมจับเข็ม needle_holder

☐ บริเวณแผล wound

☐ ปากคีบ forcep

หลักการเลือกแผนภาพให้เหมาะกับงาน

- **อย่าวาดทุกแบบเพราะเห็นในตำรา** — แผนภาพแต่ละชนิดตอบคำถามคนละข้อ ถ้าไม่มีคำถามนั้นในรายงาน ก็ไม่ต้องมีรูปนั้น ทุกรูปในเอกสารนี้จึงระบุไว้ชัดว่า “ตอบคำถามอะไร” และ “ใช้ในบทไหน”
- **ระบบนี้ไม่มีฐานข้อมูลเชิงสัมพันธ์** — จึงไม่มี ER Diagram ในความหมายของ schema ฐานข้อมูล แต่มีรูปที่ 11 อธิบายโครงสร้างข้อมูล JSON แทน พร้อมเหตุผลของการตัดสินใจนี้
- **ความซับซ้อนของรูปต้องเท่ากับความซับซ้อนของระบบจริง** — ระบบนี้ออกแบบให้ใช้เครื่องมือน้อยที่สุด (ไม่มี DB, ไม่มี job queue, ไม่มี framework ผังหน้าเว็บ) แผนภาพจึงต้องสื่อความเรียบง่ายนั้นตรง ๆ ไม่ใช่วาดให้ดูใหญ่โตเกินของจริง

หมวด ก

ภาพรวมระบบ

ระบบประกอบด้วยอะไรบ้าง ส่วนไหนคุยกับส่วนไหน และรันอยู่บนเครื่องอะไร — สองรูปนี้คือรูปเปิดของบทสถาปัตยกรรม

รูปที่ 1 แผนภาพสถาปัตยกรรมระบบ (System Architecture Diagram)

แบบแบ่งชั้น (Layered) · วาดด้วย SVG · ครอบคลุมทั้งโปรเจกต์

ตอบคำถามอะไร

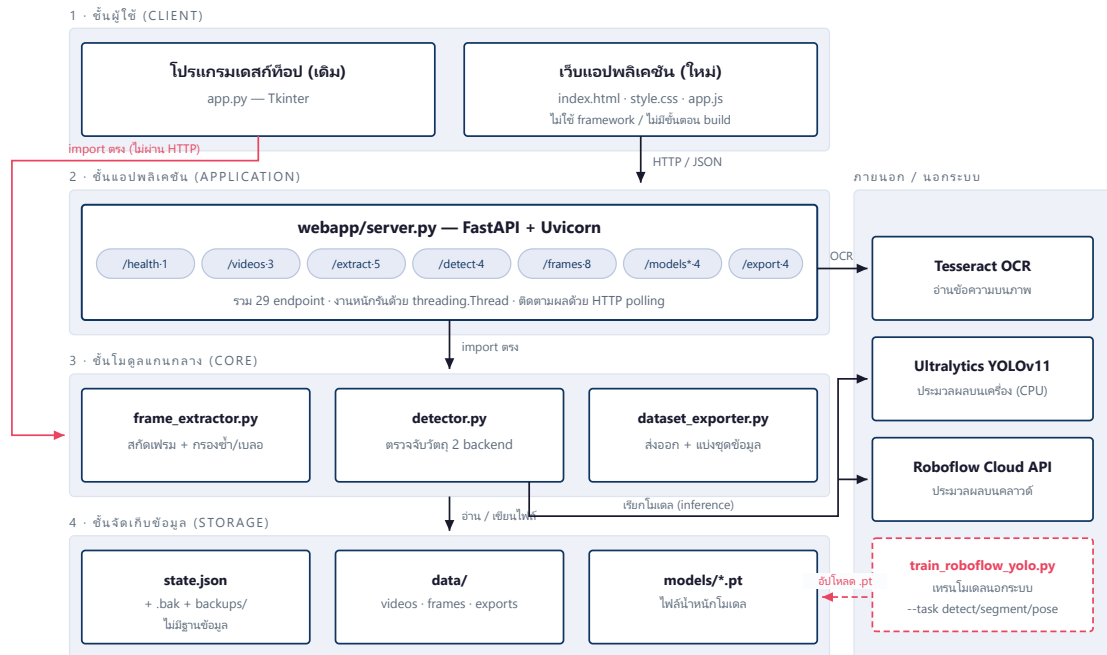
ระบบแบ่งเป็นกี่ชั้น แต่ละชั้นมีอะไร และ ทำไมโปรแกรมเดสก์ท็อปเดิมกับเว็บแอปใหม่ถึงอยู่
รวมกันได้ โดยไม่ต้องเขียนตรรกะซ้ำสองชุด — นี่คือข้อความหลักที่รูปนี้ต้องสื่อ

กล่องทึบ ส่วนประกอบที่เป็นโค้ดของโครงการเอง

กล่องเส้นประ กระบวนการนอกระบบ ทำด้วยมือเป็นครั้งคราว

ลูกศรทึบ การเรียกใช้งานโดยตรง

ลูกศรเส้นประ การส่งไฟล์เข้าระบบด้วยมือ



ข้อความหลักของรูปนี้: โปรแกรมเดสก์ท็อปเดิมและเว็บแอปใหม่เป็น “ผู้ใช้งาน” คนละตัวของ โมดูลแกนกลางชุดเดียวกัน — เดสก์ท็อป import โมดูลตรง ส่วนเว็บเรียกผ่าน HTTP มาที่ server.py ซึ่ง import โมดูลชุดเดิม จึงไม่มีตรรกะการสกัดเฟรม/ตรวจจับ/ส่งออก เขียนซ้ำสองชุด · เครื่องหมาย * ที่ /models รวม /classes และ /analytics ไว้ด้วย · การเทรนโมเดลอยู่นอกระบบ (กล่องเส้นประ) ผลลัพธ์เข้าระบบผ่านการอัปโหลดไฟล์ .pt

คำอธิบายส่วนประกอบ

ชั้นที่ 1 ผู้ใช้ — app.py คือโปรแกรม Tkinter เดิมที่ยังใช้งานคู่ขนานอยู่ ส่วนเว็บแอปเป็นไฟล์ HTML/CSS/JS อย่างละไฟล์ ไม่ใช้ framework และไม่มีขั้นตอน build

ชั้นที่ 2 แอปพลิเคชัน — webapp/server.py ไฟล์เดียว รวม 29 endpoint แบ่งเป็น 7 กลุ่ม งานที่ใช้เวลานาน (สกัดเฟรม ตรวจจับ ส่งออก) แยกไปรันใน threading.Thread แล้วให้หน้าเว็บ ถามความคืบหน้าเป็นระยะ (polling) แทนการใช้ WebSocket

ชั้นที่ 3 โมดูลแกนกลาง — สามไฟล์ที่บรรจุตรรกะจริงทั้งหมด ถูกเรียกใช้ร่วมกันจากทั้งสอง client

ชั้นที่ 4 จัดเก็บข้อมูล — ไม่มีระบบฐานข้อมูล ใช้ state.json ไฟล์เดียวเก็บสถานะทั้งหมด เขียนแบบ atomic (เขียนไฟล์ .tmp แล้วสลับชื่อ) มีสำเนาสำรอง .bak และ snapshot ตามเวลาในโฟลเดอร์ backups/

คอลัมน์ขวา — บริการที่ระบบเรียกใช้แต่ไม่ได้เขียนเอง และสคริปต์เทรนโมเดลซึ่งทำงานนอกระบบ

หลักการเขียนที่ถูกต้อง

- แบ่งชั้นตามทิศทางการพึ่งพา — ชั้นบนเรียกชั้นล่างได้ แต่ชั้นล่างต้องไม่รู้จักชั้นบน รูปนี้จึงวางลูกศรจากบนลงล่างเสมอ ถ้าพบลูกศรย้อนขึ้นแปลว่าการออกแบบมีปัญหา
- ลูกศรต้องมีป้ายกำกับ — “ลูกศรเปล่า” บอกได้แค่ “เกี่ยวข้องกัน” แต่ import ตรง กับ HTTP / JSON บอกกลไกที่ต่างกันชัดเจน ซึ่งคือสาระสำคัญของรูปนี้พอดี
- สิ่งที่ยูนอกขอบเขตระบบต้องแยกให้เห็น — ใช้เส้นประกับ `train_roboflow_yolo.py` เพื่อไม่ให้ผู้อ่านเข้าใจผิดว่าเว็บแอปเรนโมเดลตัวเอง (ซึ่งไม่จริง)
- อย่าวาดกล่องที่ไม่มีในระบบจริง — ไม่มี load balancer, message queue, หรือ database server ในรูปนี้ เพราะระบบจริงไม่มี การใส่เข้ามาให้ดูสมบูรณ์คือการรายงานเท็จ

รูปที่ 2 แผนภาพการติดตั้งใช้งาน (Deployment Diagram)

UML 2.x · วาดด้วย SVG · แสดงว่าซอฟต์แวร์ถูกติดตั้งและรันอยู่บนฮาร์ดแวร์ใด

ตอบคำถามอะไร

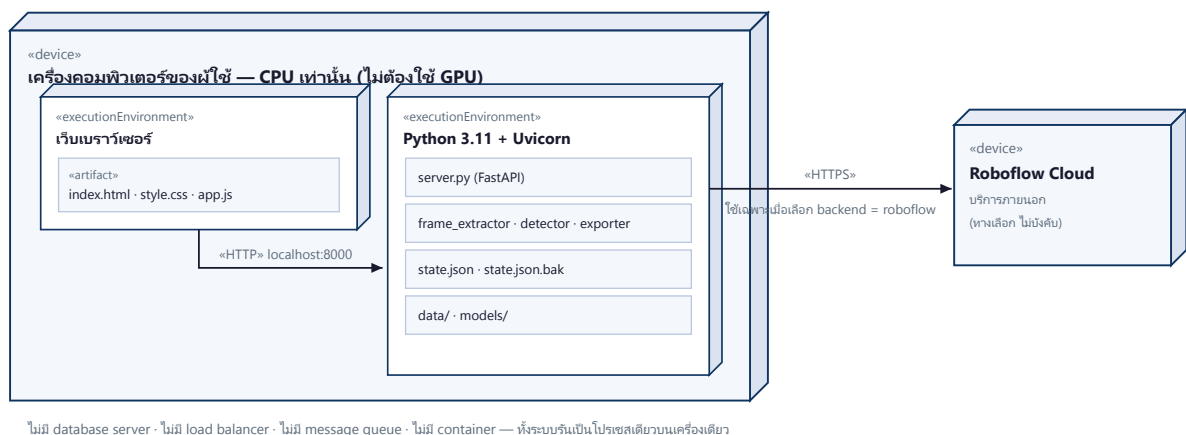
รูปที่ 1 บอกว่าระบบ ประกอบด้วย อะไร รูปนี้บอกกว่าของเหล่านั้น ไปรันอยู่ที่ไหน — คำถามที่กรรมการมักถามคือ “ต้องใช้เซิร์ฟเวอร์กี่เครื่อง ต้องใช้ GPU ไหม” รูปนี้ตอบว่า เครื่องเดียว ใช้ซีพียูอย่างเดียว

กล่องสามมิติ โหนด (Node) — อุปกรณ์หรือสภาพแวดล้อมที่รันซอฟต์แวร์ได้

«device» อุปกรณ์ทางกายภาพ

«executionEnvironment» สภาพแวดล้อมการทำงานที่อยู่ในอุปกรณ์

«artifact» ไฟล์จริงที่ถูกติดตั้งลงไป



ข้อความหลักของรูปนี้: ทั้งระบบติดตั้งบนเครื่องเดียว เบราว์เซอร์กับเซิร์ฟเวอร์อยู่ในเครื่องเดียวกัน สื่อสารผ่าน localhost การเชื่อมต่ออินเทอร์เน็ตมีเพียงเส้นเดียวคือไป Roboflow Cloud และเป็น **ทางเลือก** เท่านั้น — ถ้าเลือกประมวลผลด้วย YOLOv11 บนเครื่องระบบทำงานได้โดยไม่ต้องต่ออินเทอร์เน็ตเลย

หลักการเขียนที่ถูกต้อง

- **โน้ตต้องวาดเป็นกล่องสามมิติ** ไม่ใช่สี่เหลี่ยมแบน — ในภาษา UML สี่เหลี่ยมแบนหมายถึงคลาสหรือ คอมโพเนนต์ ส่วนกล่องสามมิติสงวนไว้สำหรับโน้ตโดยเฉพาะ
- **ต้องระบุ stereotype ในเครื่องหมาย «guillemets»** เพื่อแยกว่าโน้ตนั้นเป็นอุปกรณ์ทางกายภาพ («device») หรือสภาพแวดล้อมซอฟต์แวร์ («executionEnvironment») การเขียนแค่ชื่อเฉย ๆ ทำให้ผู้อ่านแยกไม่ออก
- **เส้นเชื่อมระหว่างโน้ตคือ communication path** ต้องกำกับโปรโทคอลเสมอ («HTTP» , «HTTPS») เพราะสาระของแผนภาพนี้อยู่ที่ “คุยกันด้วยอะไร”
- **Artifact คือไฟล์ ไม่ใช่แนวคิด** — ต้องเขียนชื่อไฟล์จริงอย่าง server.py , state.json ไม่ใช่คำอย่าง “ระบบหลังบ้าน” ซึ่งเป็นระดับคอมโพเนนต์ ไม่ใช่ระดับ deployment

หมวด ข

การไหลของข้อมูล

วิเคราะห์ระบบด้วยวิธีเชิงโครงสร้าง (Structured Analysis) — เริ่มจากมองระบบเป็นกล่องดำหนึ่งใบ แล้วค่อยเปิดออกดูข้างในทีละระดับ เป็นรูปแบบที่รายงานโครงการสายวิศวกรรมซอฟต์แวร์ในไทยใช้กันมากที่สุด

รูปที่ 3 แผนภาพบริบท (Context Diagram / DFD Level 0)

สัญลักษณ์แบบ Yourdon–DeMarco · วาดด้วย SVG

ตอบคำถามอะไร

ขอบเขตของระบบอยู่ตรงไหน — อะไรอยู่ในระบบ อะไรอยู่นอกระบบ และมีข้อมูลอะไรไหลข้ามเส้นแบ่งนั้นบ้าง รูปนี้ต้องมี **โปรเซสเดียวเท่านั้น** คือตัวระบบทั้งระบบ

วงกลม โปรเซส (Process) — ในระดับนี้มีได้เพียงวงเดียว หมายถึงเลข 0

ลูกศร กระแสข้อมูล (Data Flow) — ต้องกำกับชื่อข้อมูลเสมอ

สี่เหลี่ยม ตัวแทนภายนอก (External Entity) — คนหรือระบบที่อยู่นอกขอบเขต

ไม่มีแหล่งจัดเก็บข้อมูล ในระดับนี้ — จะปรากฏในระดับที่ 1 เท่านั้น



ข้อความหลักของรูปนี้: ระบบมีตัวแทนภายนอกเพียง 2 ตัว — ผู้ใช้งาน และ Roboflow Cloud สังเกตว่า **Tesseract OCR และ Ultralytics YOLOv11 ไม่ใช่ตัวแทนภายนอก** เพราะเป็นไลบรารีที่ติดตั้ง และรันอยู่ในระบบเอง ไม่ได้อยู่นอกขอบเขต ต่างจาก Roboflow ที่เป็นบริการบนเครือข่ายจริง ๆ

หลักการเขียนที่ถูกต้อง

- **ต้องมีโปรเซสเดียวเท่านั้น** และใช้หมายเลข 0 เสมอ — ถ้าวาดหลายวงในระดับนี้แสดงว่าสับสน กับ DFD ระดับที่ 1
- **ห้ามมีแหล่งจัดเก็บข้อมูล (Data Store)** ในแผนภาพบริบท เพราะแหล่งจัดเก็บข้อมูลอยู่ *ภายใน* ระบบ ซึ่งระดับนี้ยังถือว่าเป็นกล่องดำอยู่ นี่คือข้อผิดพลาดที่พบบ่อยที่สุดในรายงานนักศึกษา
- **ห้ามลากเส้นระหว่างตัวแทนภายนอกด้วยกันเอง** — ถ้าผู้ใช้กับ Roboflowคุยกันเองโดยไม่ผ่านระบบ เรื่องนั้นอยู่นอกขอบเขต ไม่ต้องวาด
- **กระแสข้อมูลต้องเป็นคำนาม** ("ไฟล์วิดีโอ", "ผลตรวจจับ") ไม่ใช่คำกริยา ("อัปโหลด", "ประมวลผล") เพราะสิ่งที่ไหลบนเส้นคือ *ข้อมูล* ไม่ใช่ *การกระทำ*
- **ต้องสมดุลกับระดับถัดไป (Balancing)** — กระแสข้อมูลเข้า-ออกในรูปที่ 3 ต้องปรากฏครบ ในรูปที่ 4 ห้ามมีข้อมูลโผล่มาใหม่หรือหายไปเลย ๆ

รูปที่ 4 แผนภาพกระแสข้อมูลระดับที่ 1 (Data Flow Diagram — Level 1)

สัญลักษณ์แบบ Gane-Sarson · วาดด้วย SVG · 6 โปรเซส · 5 แหล่งจัดเก็บข้อมูล

ตอบคำถามอะไร

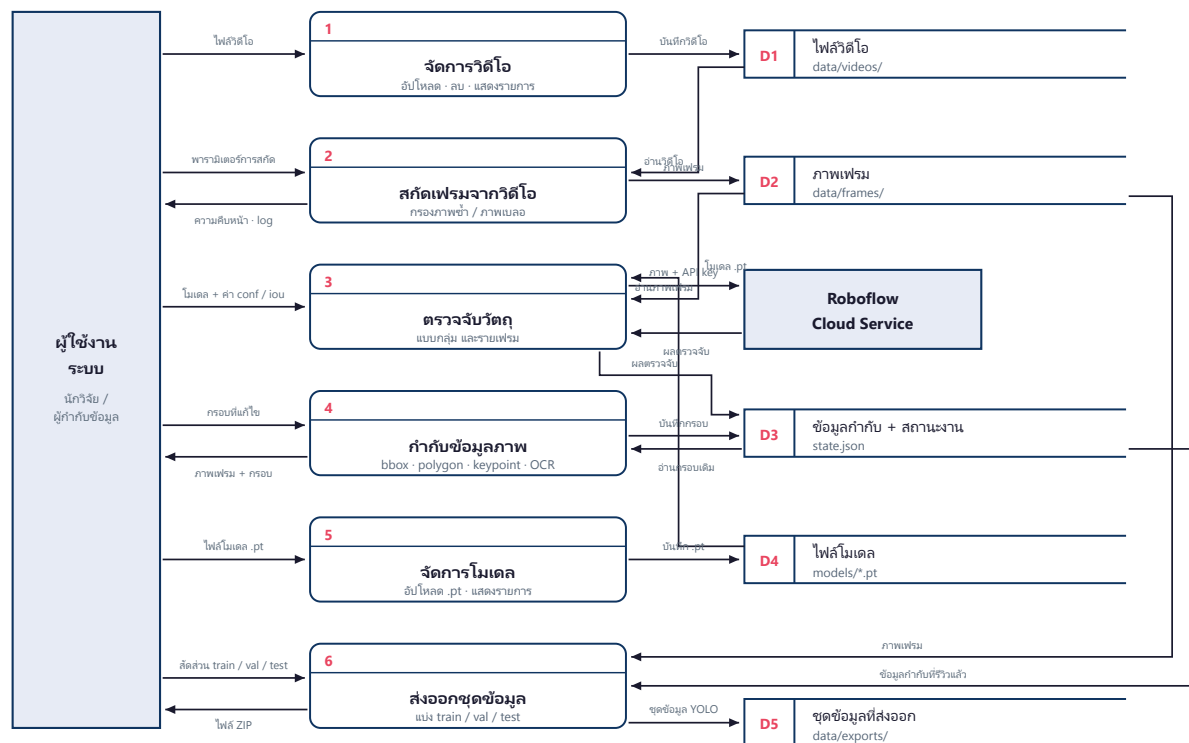
เปิดกล่องดำในรูปที่ 3 ออกมาดูข้างใน — ระบบทำงานย่อยอะไรบ้าง แต่ละงานอ่าน/เขียนข้อมูลจากที่ไหน รูปนี้คือหัวใจของบทวิเคราะห์และออกแบบระบบ และเป็นรูปที่กรรมการมักซักถามมากที่สุด

สี่เหลี่ยมมุมมน โปรเซส — มีหมายเลขกำกับ (1–6)

สี่เหลี่ยมเปิดด้านขวา แหล่งจัดเก็บข้อมูล — มีรหัส D1–D5

สื่เปลี่ยนที่บ ตัวแทนภายนอก

ลูกศร กระแสข้อมูล พร้อมชื่อข้อมูลกำกับ



ข้อความหลักของรูปนี้: state.json (D3) คือศูนย์กลางของระบบ — โพรเซส 3 (ตรวจจับ), 4 (กำกับข้อมูล) และ 6 (ส่งออก) ล้วนอ่าน/เขียนที่เดียวกัน นี่คือเหตุผลที่ไฟล์เดียวนี้นี้ต้องมีระบบสำรองข้อมูล ในเมื่อไม่มีฐานข้อมูลมาช่วย · สังเกตว่า **ไม่มีลูกศรตรงระหว่างแหล่งจัดเก็บข้อมูลสองแหล่ง** ทุกการเคลื่อนย้ายข้อมูลต้องผ่านโพรเซสเสมอ

ตรวจสอบความสมดุล (BALANCING) กับรูปที่ 3

กระแสข้อมูลในรูปที่ 3	ปรากฏที่ใดในรูปที่ 4
ไฟล์วิดีโอการผ่าตัด	ผู้ใช้งาน → โพรเซส 1
พารามิเตอร์การสกัด / ตรวจจับ	ผู้ใช้งาน → โพรเซส 2 และ โพรเซส 3
ข้อมูลกำกับที่แก้ไข	ผู้ใช้งาน → โพรเซส 4
ไฟล์โมเดล .pt	ผู้ใช้งาน → โพรเซส 5
ภาพเฟรมพร้อมกรอบวัตถุ	โพรเซส 4 → ผู้ใช้งาน
ความคืบหน้างาน	โพรเซส 2 → ผู้ใช้งาน
ไฟล์ ZIP ชุดข้อมูลที่ส่งออก	โพรเซส 6 → ผู้ใช้งาน
ภาพเฟรม + API key ↔ ผลตรวจจับ	โพรเซส 3 ↔ Roboflow Cloud Service

ทุกกระแสข้อมูลในรูปที่ 3 ปรากฏครบในรูปที่ 4 และไม่มีกระแสข้อมูลใดโผล่มาใหม่จากภายนอก

— ถือว่าสมดุล

หลักการเขียนที่ถูกต้อง

- ห้ามลากเส้นตรงระหว่างแหล่งจัดเก็บข้อมูลสองแหล่ง — ข้อมูลไม่ย้ายตัวเอง ต้องมีโพรเซสเป็นตัวอ่าน แล้วเขียนเสมอ เช่นเดียวกับที่ห้ามลากเส้นตรงระหว่างตัวแทนภายนอกกับแหล่งจัดเก็บข้อมูล
- ชื่อโพรเซสต้องเป็น “คำกริยา + กรรรม” เช่น “สกัดเฟรมจากวิดีโอ” ไม่ใช่คำนามลอย ๆ อย่าง “การสกัดเฟรม” หรือชื่อโมดูลอย่าง `frame_extractor.py` (ชื่อไฟล์เป็นเรื่องของ แผนภาพสถาปัตยกรรม ไม่ใช่ DFD)
- ทุกโพรเซสต้องมีทั้งข้อมูลเข้าและออก — โพรเซสที่มีแต่ข้อมูลเข้าเรียกว่า “หลุมดำ” (black hole) มีแต่ข้อมูลออกเรียกว่า “อัสจรรย์” (miracle) ทั้งสองแบบคือความผิดพลาดของแบบจำลอง
- เลขโพรเซสต้องสอดคล้องกับลำดับชั้น — ถ้าแตกโพรเซส 2 ต่อไปเป็นระดับที่ 2 ลูกของมันต้องเป็น 2.1, 2.2, 2.3 เสมอ
- ไม่ต้องแสดงลำดับเวลาหรือเงื่อนไข — DFD บอกว่าข้อมูลไหลไปไหน ไม่ได้บอกว่าอะไรเกิดก่อนหลัง ถ้าต้องการสื่อลำดับเวลาให้ใช้รูปที่ 6–7 (Sequence) หรือรูปที่ 8–9 (ผังงาน) แทน

หมวด ค

มุมมองผู้ใช้

ระบบทำอะไรให้ผู้ใช้ได้บ้าง เมื่อมองจากภายนอกโดยไม่สนใจว่าข้างในเขียนอย่างไร

รูปที่ 5 แผนภาพยูสเคส (Use Case Diagram)

UML 2.x · วาดด้วย SVG · 8 ยูสเคสหลัก · แสดงความสัมพันธ์ครบทั้ง 3 ชนิด

ตอบคำถามอะไร

ใครใช้ระบบได้บ้าง และใช้ทำอะไรได้บ้าง — เป็นรูปที่สื่อสารกับผู้ที่ไม่ใช่โปรแกรมเมอร์ได้ดีที่สุด และเป็นรูปที่บอกว่าเว็บแอปเพิ่มความสามารถอะไรให้โครงการบ้าง เมื่อเทียบกับระยะที่ 1 ที่ยังไม่มีเครื่องมือ

รูปคน ตัวแสดง (Actor) — ผู้ใช้หลักอยู่ซ้าย ระบบภายนอกอยู่ขวา

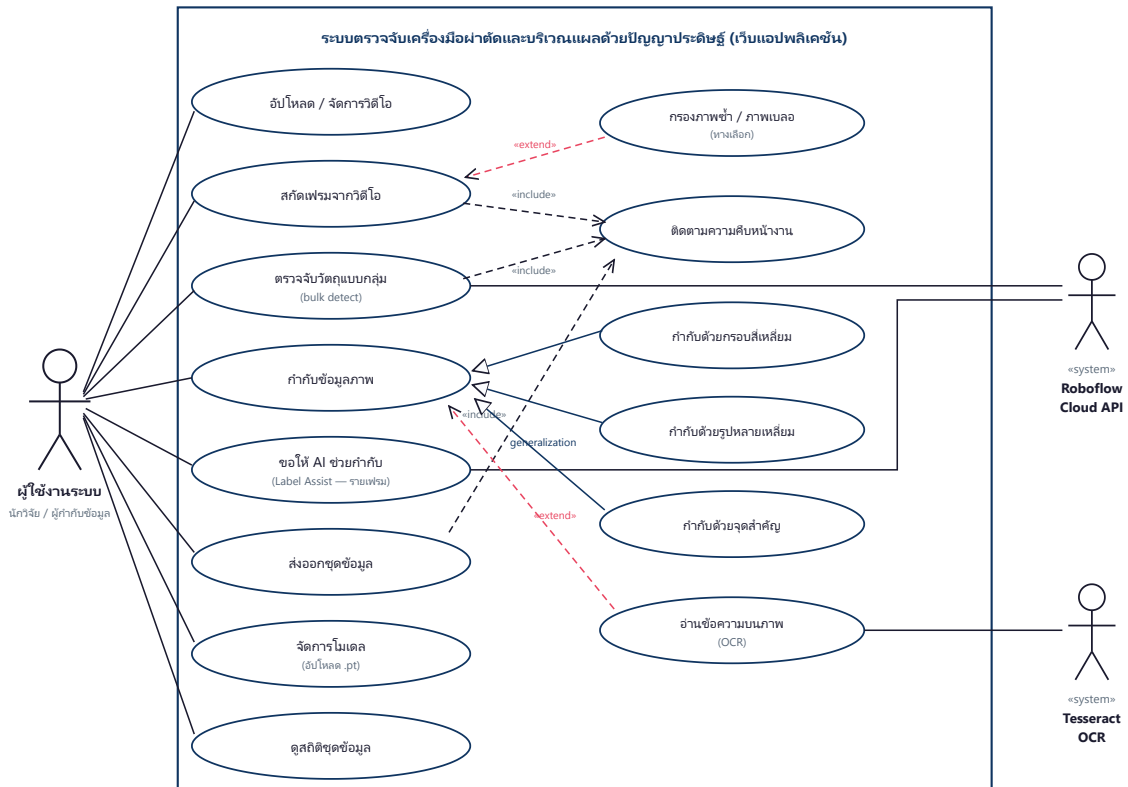
เส้นทึบไม่มีหัวลูกศร ความสัมพันธ์ (Association)

เส้นประ «extend» พฤติกรรมเสริมที่เกิดบางกรณี

วงรี ยูสเคส — เป้าหมายหนึ่งอย่างที่ผู้ใช้งานต้องการทำให้สำเร็จ

เส้นประ «include» ยูสเคสหลักเรียกใช้เสมอ

สามเหลี่ยมโปรง การสืบทอด (Generalization)



ข้อความหลักของรูปนี้: ผู้ใช้มีเป้าหมายหลัก 8 อย่าง โดย “กำกับข้อมูลภาพ” มีรูปแบบย่อย 3 ชนิด (สืบทอดกัน) และมีพฤติกรรมเสริมคือ OCR · “ติดตามความคืบหน้างาน” ถูกเรียกใช้เสมอโดยยูสเคสที่เป็นงานเบื้องหลัง ทั้งสาม จึงแยกออกมาเป็นยูสเคสร่วมด้วย «include» แทนการเขียนซ้ำสามที่

ทำไมบางอย่างถึงไม่ได้เป็นยูสเคส

“ยืนยัน / ปฏิเสธผลจาก AI” — เป็น **ขั้นตอนหนึ่ง** ภายในการกำกับข้อมูลภาพ ไม่ใช่เป้าหมายที่ผู้ใช้ เปิดระบบขึ้นมาเพื่อทำ จึงอยู่ในคำบรรยายยูสเคส (Use Case Description) ไม่ใช่วงรีของตัวเอง

“เข้าสู่ระบบ” — เวอร์ชันปัจจุบันตัดระบบยืนยันตัวตนออกโดยตั้งใจ เพื่อให้ใช้งานภายในเครื่องได้ทันที จึงไม่มียูสเคสนี้ ถ้าใส่เข้ามาจะเป็นการรายงานสิ่งที่ไม่จริง

“เทรนโมเดล” — ทำด้วยสคริปต์นอกระบบ ไม่ได้ทำผ่านเว็บ จึงอยู่นอกกรอบขอบเขตระบบ สิ่งที่อยู่ในระบบคือ “จัดการโมเดล” (อัปโหลดไฟล์ผลลัพธ์เข้ามา) เท่านั้น

หลักการเขียนที่ถูกต้อง

- ยูสเคสต้องเป็นเป้าหมายที่ผู้ใช้ได้คุณค่า ไม่ใช่ขั้นตอนย่อย — “ส่งออกชุดข้อมูล” เป็นยูสเคส แต่ “กดปุ่มบันทึก” หรือ “ตรวจสอบความถูกต้องของฟอร์ม” ไม่ใช่
- เส้น **Association** ไม่มีหัวลูกศร และไม่ใช่กระแสข้อมูล — เป็นข้อผิดพลาดที่พบบ่อยมาก คนมักวาดลูกศรแล้วเขียนว่า “ส่งข้อมูล” ซึ่งเป็นการเอา DFD มาปนกับ UML
- ทิศทางของ «include» กับ «extend» ตรงข้ามกัน — «include» ชี้ จาก ยูสเคสหลัก ไป ยูสเคสที่ถูกเรียกใช้เสมอ ส่วน «extend» ชี้ จาก ยูสเคสเสริม ไป ยูสเคสหลักที่ถูกขยาย จำผิดทิศเป็นข้อผิดพลาดที่กรรมการจับได้ทันที

- **ตัวแสดงไม่จำเป็นต้องเป็นคน** — ระบบภายนอกที่ระบบเราต้องคุยด้วยก็เป็นตัวแสดง (secondary actor) นิยมวางไว้ด้านขวา และกำกับด้วย «system» เพื่อไม่ให้เข้าใจผิดว่าเป็นมนุษย์
- **กรอบระบบต้องมีจริงและมีชื่อ** — สิ่งที่อยู่ในกรอบคือสิ่งที่เราสร้าง สิ่งอยู่นอกกรอบคือสิ่งที่เราไม่ได้สร้าง

หมวด ง

พฤติกรรมเชิงเวลา

รูปก่อนหน้าบอกโครงสร้างและขอบเขต แต่ไม่บอกว่า “เกิดอะไรก่อนหลัง” — สองรูปนี้เจาะจงสองสถานการณ์ ที่ออกแบบต่างกันโดยตั้งใจ เพื่อแสดงผลของการเลือกใช้งานเบื้องหลังแบบไม่พร้อมกัน (asynchronous) กับแบบรอผลทันที (synchronous)

รูปที่ 6 แผนภาพลำดับ: งานสัปดาห์ (Sequence Diagram — Asynchronous Job)

UML 2.x · วาดด้วย Mermaid · แสดง Combined Fragment ชนิด par และ loop

ตอบคำถามอะไร

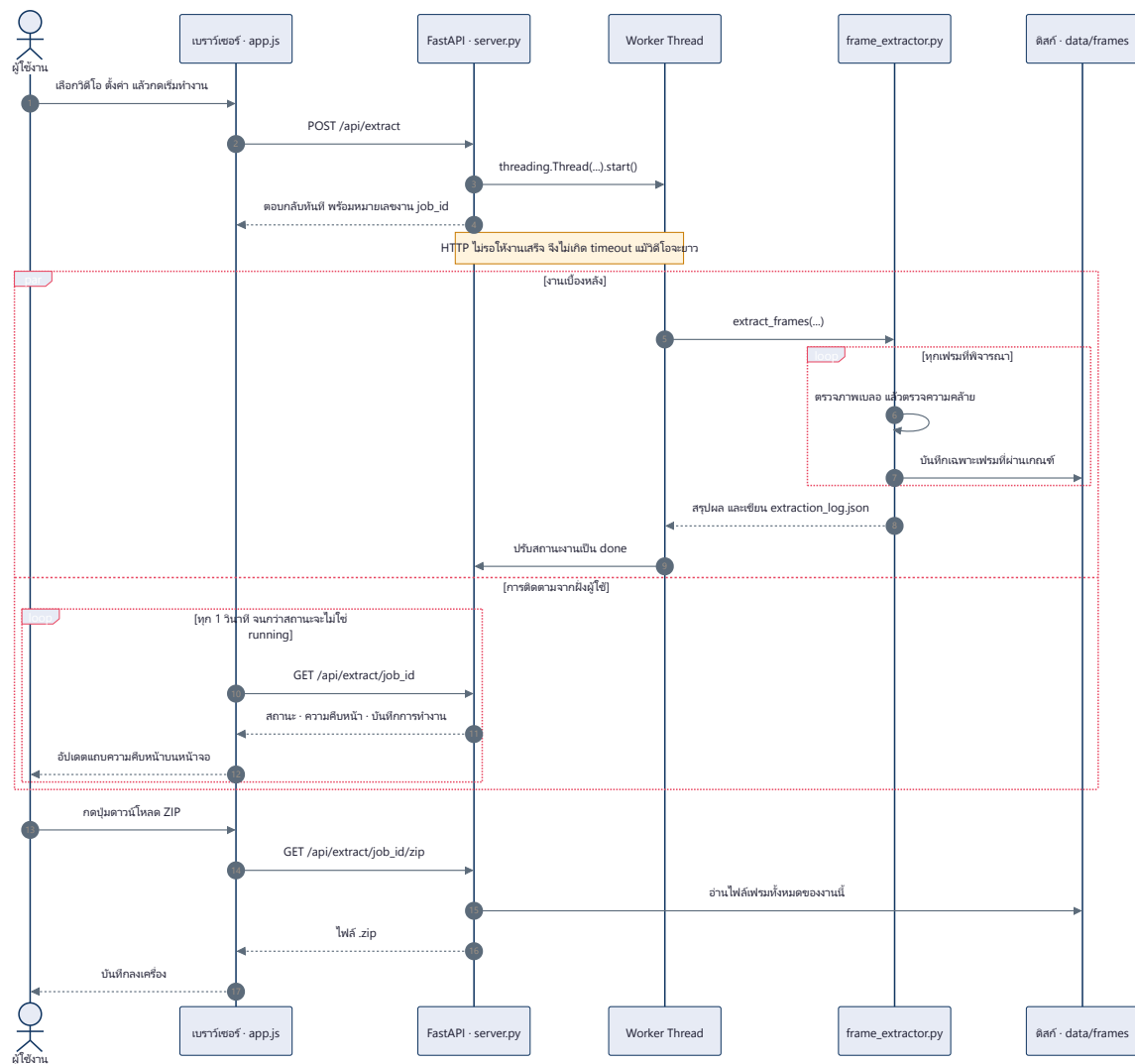
งานสัปดาห์จากวิดีโอความยาวหลายนาทีใช้เวลานานกว่าที่คำขอ HTTP หนึ่งครั้งจะรอไหว รูปนี้แสดงว่าระบบแก้ปัญหาได้อย่างไร — **ตอบกลับทันทีพร้อมหมายเลขงาน แล้วให้หน้าเว็บถามความคืบหน้าเป็นระยะ**

เส้นประแนวตั้ง เส้นชีวิต (Lifeline) ของแต่ละส่วนประกอบ

แท่งสีเหลี่ยม ช่วงที่กำลังทำงาน (Activation / Execution Occurrence)

ลูกศรเส้นทึบ การเรียกใช้ · **ลูกศรเส้นประ** การตอบกลับ

กรอบ par / loop ทำงานพร้อมกัน / ทำซ้ำ



ข้อความหลักของรูปนี้: ข้อความที่ 4 คือหัวใจ — เซิร์ฟเวอร์ตอบกลับ *ก่อน* งานจะเสร็จ ทำให้ฝั่งซ้าย (ผู้ใช้) กับฝั่งขวา (งานเบื้องหลัง) เดินคู่ขนานกันไป และเป็นเหตุผลที่ระบบเลือกใช้ การถามซ้ำเป็นระยะ (polling) แทน WebSocket — ได้ผลเหมือนกันในการใช้งานคนเดียว แต่ไม่ต้องดูแล วงจรชีวิตของการเชื่อมต่อเลย

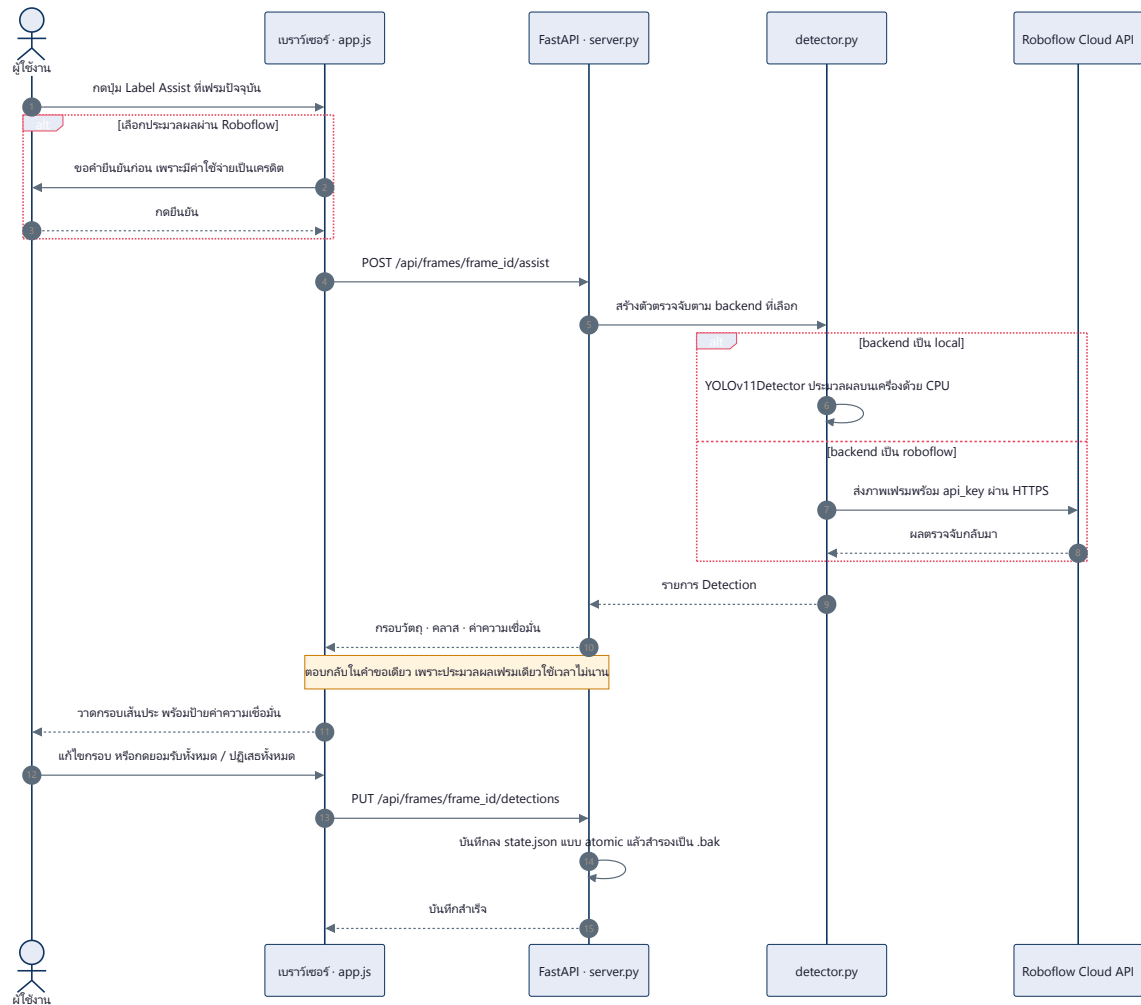
รูปที่ 7 แผนภาพลำดับ: การขอให้ AI ช่วย

กำกับ (Sequence Diagram — Synchronous)

UML 2.x · วาดด้วย Mermaid · แสดง Combined Fragment ชนิด alt

ตอบคำถามอะไร

ทำไมงานตรวจจับ **ทีละเฟรม** ถึงไม่ต้องใช้กลไกงานเบื้องหลังเหมือนรูปที่ 6 และ **ระบบเลือกใช้** โมเดลบนเครื่องหรือบนคลาวด์อย่างไร รวมถึงกลไกป้องกันการใช้เครดิตโดยไม่ตั้งใจ



ข้อความหลักของรูปนี้: กรอบ alt อันแรกคือมาตรการป้องกันเครดิต — ระบบถามยืนยันก่อน ยิงค่าขอออกคราวดีทุกครั้ง · กรอบ alt อันที่สองแสดงว่าโค้ดเรียกใช้ detector.py แบบเดียวกันเสมอ ความต่างของ backend ถูกซ่อนไว้หลังคลาสฐาน BaseDetector ส่วนที่เรียกใช้จึงไม่ต้องรู้ว่าเบื้องหลังประมวลผลที่ไหน

หลักการเขียนที่ถูกต้อง (ใช้กับทั้งรูปที่ 6 และ 7)

- เวลาไหลจากบนลงล่างเสมอ — ตำแหน่งแนวตั้งของข้อความสื่อลำดับก่อนหลัง ห้ามวางข้อความที่เกิดทีหลัง ไว้สูงกว่าข้อความที่เกิดก่อน
- แยกลูกศรเรียกกับลูกศรตอบให้ชัด — เส้นทึบหัวลูกศรทึบคือการเรียกแบบรอผล เส้นประคือการตอบกลับ การใช้เส้นทึบทั้งหมดทำให้ดูไม่ออกว่าใครรอใคร ซึ่งเป็นสาระสำคัญของรูปที่ 6 พอดี
- ใช้ Combined Fragment แทนการวาดหลายรูป — สื่อนไขใช้ alt , การทำซ้ำใช้ loop , การทำงานพร้อมกันใช้ par ไม่ต้องแยกเป็นรูปละครณี
- หนึ่งรูปต่อหนึ่งสถานการณ์ — Sequence Diagram อธิบาย สถานการณ์หนึ่ง ไม่ใช่ทั้งระบบ การพยายามยัดทุกฟีเจอร์ลงรูปเดียวทำให้อ่านไม่รู้เรื่องและผิดวัตถุประสงค์ของแผนภาพชนิดนี้
- ชื่อเส้นชีวิตต้องเป็นวัตถุหรือส่วนประกอบ ไม่ใช่การกระทำ — server.py ใช้ได้ แต่ “การประมวลผล” ใช้ไม่ได้

หมวด จ

อัลกอริทึมและกระบวนการ

สองรูปนี้ลงลึกถึงตรรกะจริง — รูปแรกคืออัลกอริทึมภายในโค้ด รูปที่สองคือกระบวนการทำงานของมนุษย์ ที่ใช้เว็บแอปเป็นเครื่องมือ ซึ่งเป็นสิ่งที่รายงานระยะที่ 1 ยังไม่มี

รูปที่ 8 แผนผังอัลกอริทึมการสกัดเฟรม (Flowchart)

สัญลักษณ์มาตรฐาน ANSI/ISO 5807 • วาดด้วย Mermaid • อ้างอิง frame_extractor.py

ตอบคำถามอะไร

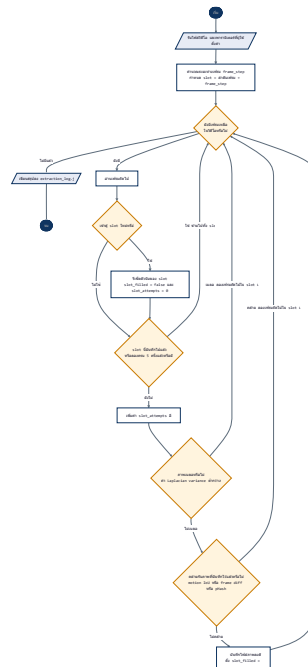
ระบบเลือกเฟรมอย่างไรให้ได้ภาพที่ **ไม่เบลอและไม่ซ้ำกัน** — โดยเฉพาะกลไก **“ถอยไปหาเฟรมถัดไปในช่วงเดิม” (slot fallback)** ซึ่งเป็นจุดที่อธิบายด้วยข้อความยาก แต่เห็นชัดเจนที่สุดในผังงาน

วงรีมน จุดเริ่มต้น / สิ้นสุด (Terminator)

สี่เหลี่ยมผืนผ้า การประมวลผล (Process)

สี่เหลี่ยมด้านขนาน ข้อมูลเข้า / ออก (Input–Output)

สี่เหลี่ยมข้าวหลามตัด การตัดสินใจ (Decision) — ต้องมีทางออกอย่างน้อย 2 ทาง



ข้อความหลักของรูปนี้: เมื่อเฟรมไม่ผ่านเกณฑ์ ระบบ **ไม่ทิ้งทั้งเวลานั้น** แต่ย้อนกลับไปจุดตัดสินใจ “ยังมีเฟรมเหลือหรือไม่” เพื่อลองเฟรมถัดไปใน slot เดิม สูงสุด 5 ครั้ง — นี่คือเหตุผลที่ ผลการสกัดจริงมักได้ภาพน้อยกว่าเป้าหมายเล็กน้อย (เช่น ตั้งไว้ 12 ได้จริง 11) ซึ่งเป็นพฤติกรรมที่ตั้งใจ ไม่ใช่ข้อผิดพลาด

หลักการเขียนที่ถูกต้อง

- ทุกผังงานต้องมีจุดเริ่มต้นเดียวและจุดสิ้นสุดที่ชัดเจน — ผังงานที่ไม่มีทางออกคือลูปไม่รู้จบ
- สัญลักษณ์ตัดสินใจต้องมีทางออกอย่างน้อย 2 ทาง และต้องกำกับเงื่อนไขทุกเส้น — เส้นที่ไม่มีป้าย ทำให้ผู้อ่านเดาไม่ออกว่าเป็นกรณีใด
- ห้ามใช้สีเหลี่ยมผืนผ้าแทนการตัดสินใจ — รูปทรงคือความหมาย ไม่ใช่การตกแต่ง การเลือกรูปทรงผิดทำให้ผังงานสื่อสารผิด
- เขียนในระดับตรรกะ ไม่ใช่ระดับโค้ด — ควรอ่านรู้เรื่องแม้ไม่รู้ภาษา Python แต่ยังคงชื่อตัวแปรสำคัญไว้ได้ เพื่อให้เทียบกับซอร์สโค้ดจริงได้

รูปที่ 9 แผนภาพกิจกรรม: วงจรกำกับข้อมูลโดยมี AI ช่วย (Activity Diagram)

UML 2.x · วาดด้วย Mermaid · แสดงกระบวนการทำงานแบบวนรอบ

ตอบคำถามอะไร

เว็บแอปเปลี่ยนวิธีทำงานของโครงการอย่างไร — ระยะที่ 1 กำกับข้อมูล 120 ภาพด้วยมือทั้งหมด แต่กระบวนการใหม่นี้ใช้โมเดลรุ่นก่อนหน้ามาช่วยกำกับข้อมูลรุ่นถัดไป แล้ววนรอบเพื่อขยายชุดข้อมูล ให้ใหญ่ขึ้นเรื่อย ๆ ด้วยแรงคนที่เท่าเดิม ซึ่งตอบโจทย์ข้อจำกัดหลักที่รายงานระยะที่ 1 ระบุไว้ คือ “ข้อมูลน้อยและความหลากหลายไม่ครอบคลุม”



ข้อความหลักของรูปนี้: กิจกรรมที่เน้นกรอบสีแดงคือจุดที่สำคัญที่สุดของทั้งวงจร — การตั้งค่า “ข้ามเฟรมที่รีวไปแล้ว” ถ้าไม่มีตัวเลือกนี้ การรันตรวจจับซ้ำด้วย โมเดลรุ่นใหม่จะเขียนทับ งานที่คนแก้ด้วยมือไปแล้วโดยไม่มีคำเตือน ทำให้วงจรนี้ทำซ้ำไม่ได้เลย · สังเกตว่ากิจกรรม “เทรนโมเดล” อยู่ในกระบวนการ แต่ทำนอกเว็บแอป ตรงกับที่แสดงเป็นกล่องเส้นประในรูปที่ 1

หลักการเขียนที่ถูกต้อง

- **สัญลักษณ์เริ่มต้นและสิ้นสุดต่างกัน** — ในมาตรฐาน UML จุดเริ่มต้นคือ วงกลมทึบ ● ส่วนจุดสิ้นสุดคือ วงกลมทึบมีวงแหวนล้อม ◎ (รูปนี้แสดงเป็นวงกลมมีข้อความกำกับตามข้อจำกัด ของเครื่องมือที่ใช้วาด เมื่อนำไปวาดในรายงานควรใช้สัญลักษณ์มาตรฐาน)
- **ชื่อกิจกรรมต้องขึ้นต้นด้วยคำกริยา** — “ส่งออกชุดข้อมูล” ไม่ใช่ “ชุดข้อมูลที่ส่งออก”
- **แผนภาพกิจกรรมต่างจากผังงานตรงที่แสดงการทำงานพร้อมกันได้** ด้วยแถบ fork/join ถ้ากระบวนการเป็นเส้นตรงล้วนอย่างรูปนี้ การใช้ผังงานธรรมดาก็เพียงพอ — เลือกใช้แผนภาพกิจกรรมที่นี่ เพราะเป็นกระบวนการระดับธุรกิจ/การทำงานของคน ไม่ใช่ตรรกะในโค้ด
- **เส้นย้อนกลับต้องกำกับเงื่อนไขเสมอ** — เส้นจาก “ยังไม่ถึงเกณฑ์” กลับไปยังขั้นส่งออก คือสิ่งที่ทำให้รูปนี้เป็น “วงจร” ไม่ใช่ “รายการขั้นตอน”

หมวด จ

ข้อมูลและสถานะ

สองรูปสุดท้ายอธิบายสิ่งที่ระบบ “จำ” ไว้ — สถานะที่เปลี่ยนไปตามเวลา และโครงสร้างของข้อมูลที่เก็บจริง

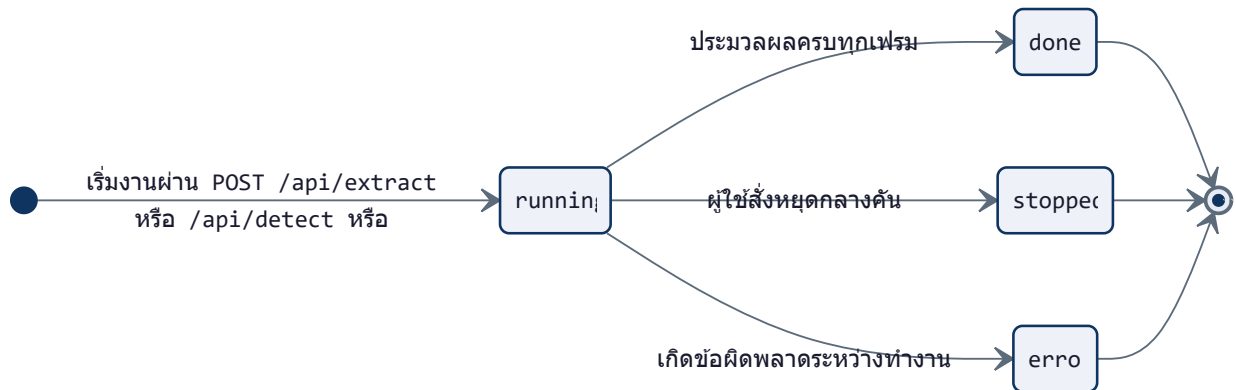
รูปที่ 10 แผนภาพสถานะ (State Machine Diagram)

UML 2.x · วาดด้วย Mermaid · แสดง 2 วงจรสถานะที่สำคัญของระบบ

ตอบคำถามอะไร

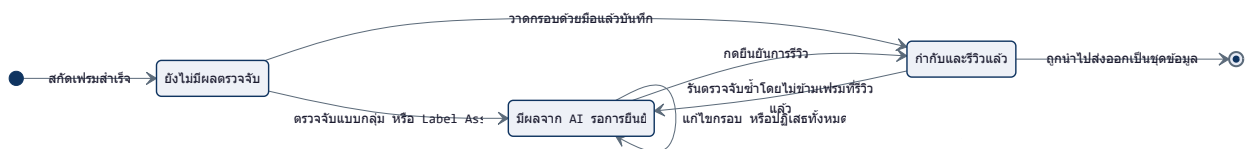
สิ่งของในระบบมีได้กี่สถานะ และเปลี่ยนสถานะเมื่อไหร่ — สำคัญเป็นพิเศษกับ **เฟรม** เพราะเป็นตัวกำหนดว่าเฟรมใดพร้อมนำไปเทรนแล้วบ้าง และเป็นที่มาของตัวเลือก “ข้ามเฟรมที่รีวิวแล้ว” ในรูปที่ 9

10.1 วงจรสถานะของงานเบื้องหลัง (JOB)



งานทุกชนิดใช้ชุดสถานะเดียวกัน · **ไม่มีสถานะรอดคิว (queued)** เพราะระบบสร้างเธรดและเริ่มทำงานทันที ไม่มีระบบคิวงาน — ตรงกับการออกแบบที่ใช้เครื่องมือน้อยที่สุด · สถานะ **stopped** ต่างจาก **error** ตรงที่เฟรมที่บันทึกไปแล้วยังใช้งานได้ตามปกติ

10.2 วงจรสถานะของเฟรม (FRAME)



ข้อความหลักของรูปนี้: เส้นเปลี่ยนสถานะจาก “กำกับและรีวิวแล้ว” ย้อนกลับไปเป็น “รอการยืนยัน” คือความเสี่ยงที่ต้องควบคุม — เป็นเส้นทางที่ทำให้งานที่คนตรวจแล้วถูกเขียนทับ ระบบจึงตั้งค่าเริ่มต้นให้ **ข้ามเฟรมที่รีวิวแล้วเสมอ** เพื่อไม่ให้เดินเส้นทางนี้โดยไม่ตั้งใจ

หลักการเขียนที่ถูกต้อง

- สถานะต้องเป็นคำนามหรือคำคุณศัพท์ ไม่ใช่คำกริยา — “กำกับและรีวิวแล้ว” คือสถานะ ส่วน “กำลังกำกับข้อมูล” เป็นกิจกรรม ควรอยู่ในแผนภาพกิจกรรมแทน
- เส้นเปลี่ยนสถานะต้องกำกับเหตุการณ์ที่ทำให้เปลี่ยน ไม่ใช่ปล่อยเส้นเปล่า — เพราะสาระของแผนภาพนี้คือ “อะไรทำให้เปลี่ยน”
- ต้องมีสถานะเริ่มต้นและสถานะสิ้นสุด และทุกสถานะต้องมีทางไปต่อ ไม่มีสถานะที่เข้าแล้วออกไม่ได้ (นอกจากตั้งใจให้เป็นสถานะสิ้นสุด)

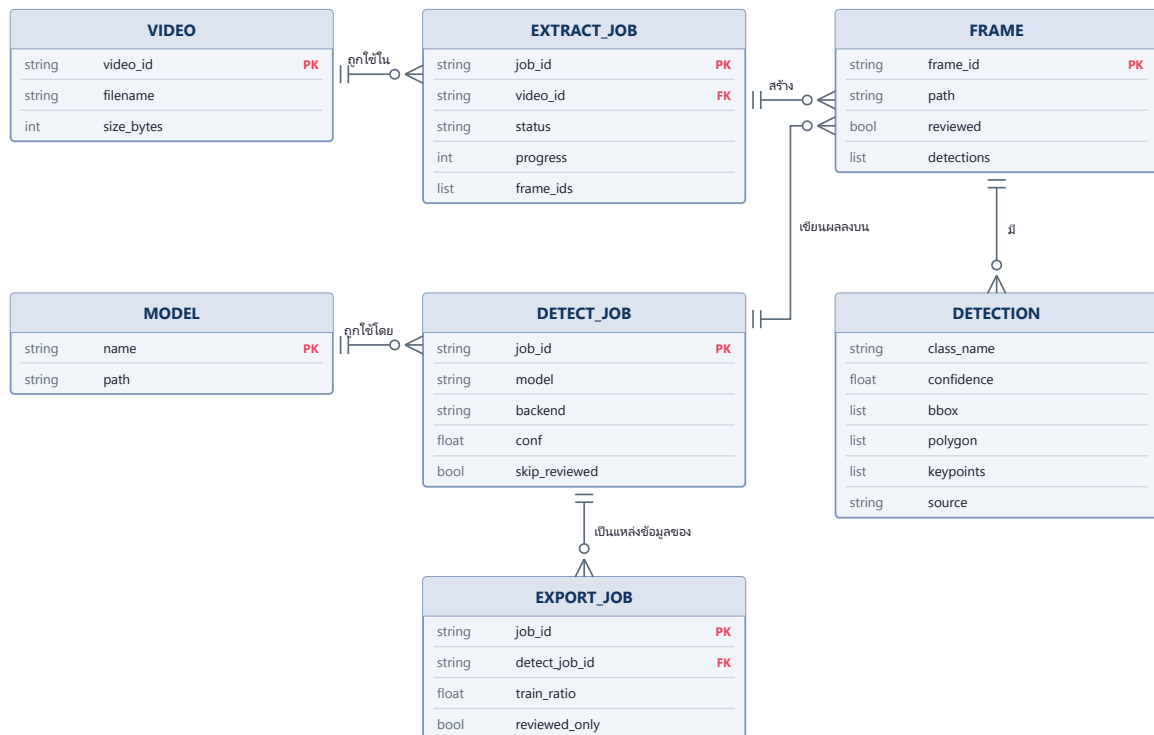
- วาดเฉพาะสถานะที่มีอยู่จริงในโค้ด — รูปที่ 10.1 จึงไม่มี `queued` , `paused` หรือ `retrying` เพราะระบบไม่มีสถานะเหล่านั้น

รูปที่ 11 แผนภาพโครงสร้างข้อมูล (Data Model Diagram)

รูปแบบ Entity-Relationship · วาดด้วย SVG · อ้างอิงโครงสร้างจริงของ `state.json`

ข้อควรระวังให้ชัดในรายงาน

- ระบบนี้ **ไม่มีฐานข้อมูลเชิงสัมพันธ์** — ข้อมูลทั้งหมดเก็บในไฟล์ JSON ไฟล์เดียวคือ `state.json` รูปนี้จึงเป็นแบบจำลองเชิงแนวคิด (**Conceptual Data Model**) ที่ยืมสัญลักษณ์ ER มาใช้อธิบายความสัมพันธ์ของข้อมูลเท่านั้น **ไม่ใช่ schema** ของฐานข้อมูลจริง ถ้าเขียนในรายงานโดยไม่บอกข้อนี้ กรรมการจะเข้าใจว่ามีฐานข้อมูล ซึ่งไม่ตรงกับระบบจริง
- **เหตุผลของการตัดสินใจ** — ระบบออกแบบให้ใช้งานทีละคน ข้อมูลอยู่ในระดับหลักสิบถึงหลักร้อยกิโลไบต์ การเพิ่มฐานข้อมูลจะพาเครื่องมือใหม่เข้ามาหลายตัว (ตัวฐานข้อมูล, ORM, ระบบย้ายโครงสร้าง) โดยไม่ได้แก้ปัญหามีอยู่จริง จึงเลือกใช้ไฟล์ JSON พร้อมกลไกเขียนแบบ atomic และสำรองข้อมูลแทน



⎓ —> <—> อ่านว่า “หนึ่ง ต่อ ศูนย์หรือมากกว่า” —> ซัดสองซัดคือหนึ่ง · วงกลมคือศูนย์ได้ · ตันภาคคือมากกว่าหนึ่ง

ข้อความหลักของรูปนี้: FRAME คือศูนย์กลาง — เป็นทั้งผลผลิตของงานสกัดเฟรม เป้าหมายของงานตรวจจับ และวัตถุดิบของงานส่งออก · ฟิลด์ source ใน DETECTION เป็นตัวแยกว่ากรอบนั้นมาจากคนวาดหรือ AI เสนอ ซึ่งเป็นข้อมูลที่ทำให้ วงจรในรูปที่ 9 ทำงานได้ · ฟิลด์ polygon และ keypoints รองรับการทำงานกับ แบบรูปหลายเหลี่ยมและจุดสำคัญตามรูปที่ 5

หลักการเขียนที่ถูกต้อง

- ระบุคีย์หลักและคีย์นอกให้ชัด (PK / FK) แม้จะเป็นแบบจำลองเชิงแนวคิด เพราะเป็นสิ่งที่บอกว่าข้อมูลอ้างอิงกันอย่างไร
- สัญลักษณ์ตีนกา (Crow's Foot) ต้องสื่อจำนวนความสัมพันธ์ให้ถูก — ||--o{ หมายถึง “หนึ่งต่อศูนย์หรือมากกว่า” เช่น งานสกัดหนึ่งงานสร้างเฟรมได้หลายเฟรม แต่เฟรมหนึ่งเฟรมมาจากงานสกัดเพียงงานเดียว
- ตั้งชื่อความสัมพันธ์เป็นคำกริยา ที่อ่านจากซ้ายไปขวาแล้วได้ประโยคสมบูรณ์ เช่น “EXTRACT_JOB สร้าง FRAME”
- ถ้าไม่มีฐานข้อมูล ต้องบอกตรง ๆ — การนำเสนอ ER Diagram โดยไม่ระบุว่าเป็นแบบจำลอง เชิงแนวคิด ถือเป็นการให้ข้อมูลที่ทำให้เข้าใจผิดเกี่ยวกับสถาปัตยกรรมจริงของระบบ

ภาคผนวก

ใช้รูปไหนในบทใดของรายงาน

ข้อเสนอการจัดวางแผนภาพทั้ง 11 รูปลงในโครงรายงานโครงการแบบมาตรฐาน

บท	รูปที่	เหตุผล
บทที่ 1 บทนำ	1	ใช้แผนภาพสถาปัตยกรรมเป็นรูปเปิด เพื่อให้ผู้อ่านเห็นภาพรวมทั้งระบบก่อนลงรายละเอียด
บทที่ 3 วิเคราะห์และออกแบบระบบ	3, 4, 5, 11	เรียงตามลำดับการวิเคราะห์มาตรฐาน — กำหนดขอบเขต (Context) → แด็กกระบวนการ (DFD ระดับ 1) → มุมมองผู้ใช้ (Use Case) → โครงสร้างข้อมูล เป็นบทที่تمرการซักถามมากที่สุด
บทที่ 4 การพัฒนาระบบ	2, 6, 7, 8, 10	ส่วนที่ลงลึกถึงการทำงานจริง — สภาพแวดล้อมที่ติดตั้ง ลำดับการทำงานสองรูปแบบ อัลกอริทึมการคัดเลือกเฟรม และการจัดการสถานะ
บทที่ 5 ผลและการอภิปราย	9	ใช้ประกอบการอภิปรายว่าเครื่องมือที่พัฒนาขึ้นช่วยแก้ข้อจำกัดหลักจากระยะที่ 1 (ข้อมูลน้อยและไม่หลากหลาย) ได้อย่างไร

ข้อควรระวังก่อนนำไปใช้

- ตัวเลขผลการทดลอง (mAP@50 68.1% · Precision 80.5% · Recall 73.1%) มาจากรายงานระยะที่ 1 เป็นผลจากการทดลองที่ทำ นอก เว็บแอปนี้ ไม่ใช่ตัวเลขที่ระบบคำนวณสด ต้องระบุที่มาให้ชัดในรายงาน

- แผนภาพทั้งหมดสะท้อนสถานะระบบ ณ เดือนสิงหาคม 2569 ถ้าพัฒนาต่อจนโครงสร้างเปลี่ยน ต้องปรับรูปตาม โดยเฉพาะจำนวน endpoint ในรูปที่ 1 และรายการยูสเคสในรูปที่ 5
- สิ่งที่ยังไม่ได้ทำ ไม่ควรปรากฏในแผนภาพ — เช่น การเชื่อมกล่องแบบเรียลไทม์ และโมเดล RNN วิเคราะห์ลำดับการเคลื่อนไหว ยังเป็นแผนในอนาคต ควรเขียนไว้ในหัวข้อ “แนวทางการพัฒนาต่อ” ไม่ใช่วาดปนลงในแผนภาพระบบปัจจุบัน